

Brief técnico — App Android para analizador facial

MJ-008

Este documento contiene todo lo necesario para construir la app. La información proviene de ingeniería inversa de interoperabilidad del APK original (`com.ym.smart.skins`), del análisis de sus librerías nativas y de los logs de la tablet. **Todo lo que aparece aquí está verificado**: no inventes valores ni “mejores” los números; el hardware no perdona aproximaciones.

Contexto: el propietario compró la máquina y tiene licencia activa del motor de análisis. La app la usará una médico en consulta. El objetivo es reemplazar la app original (inestable e inutilizable desde Venezuela por depender de servidores en China), no clonarla.

1. Hardware

Un solo dispositivo USB hace todo:

Dato	Valor
VID:PID	<code>0x0EDC:0x3080</code> (decimal 3804 / 12416)
Clase	UVC 1.00 estándar, multi-interfaz
Nodo	<code>/dev/video0</code> , cámara externa <code>device@3.4/external/100</code>
Formatos	MJPEG y YUYV hasta 3840×2160
Resolución de captura	1200×1600 (la que usa el original)
Conexión	USB 3.0

Las 8 luces cuelgan de una Extension Unit dentro de esa misma interfaz USB. No hay puerto serial, no hay segundo dispositivo, no hay GPIO.

Hay también un táctil ILITEK-TP (`vid=8746 pid=1`) en el bus: hay que excluirlo del descubrimiento.

2. REGLA DE ORO — una sola conexión USB

Este es el error que ya hizo fallar un intento anterior. Léelo dos veces.

Como las luces y la cámara son el mismo dispositivo USB, **solo puede existir una `UsbDeviceConnection` en toda la app**, y debe ser la que abre la librería UVC.

Prohibido:

- Llamar a `usbManager.openDevice()` fuera de `USBMonitor`.
- Llamar a `claimInterface(iface, true)` por tu cuenta. El `force = true` desprende a quien tenga la interfaz y rompe el stream de video.
- Tener un controlador de luces con su propio ciclo `open()` / `close()`.
- Lanzar un segundo flujo de permisos USB en paralelo al de `USBMonitor`.

Síntoma cuando se viola: la cámara no abre, preview en negro, o cuelgue en `AbstractUVCCameraHandler$CameraThread.handleOpen`.

Arquitectura correcta:

```
USBMonitor → onConnect(UsbControlBlock)
              ↓
          UVCCameraHandler.open() ← única conexión al dispositivo
              ↓
          controlLed(...) ← las luces reutilizan ESE handle
```

Orden obligatorio: **primero abrir cámara, después tocar luces**. Nunca antes de `onConnect`. Si `cameraHandler.isOpened()` es falso, no intentes encender nada.

3. Protocolo de luces (Extension Unit UVC)

Cada cambio de luz son **dos control transfers** consecutivos de 8 bytes.

Parámetros comunes a ambos:

Campo	Valor
<code>bmRequestType</code>	<code>0x21</code> (Host→Device, Class, Interface)
<code>bRequest</code>	<code>0x01</code> (SET_CUR)

wIndex	0x0400 (Unit ID 4 << 8 interfaz 0)
wLength	8
timeout	1000 ms

Paquete 1 — COMANDO (wValue = 0x0A00)

Con opcode = 0x0082 , addr = 0x00D816 , len = 4 :

```

byte[0] = (opcode >> 8) & 0xFF = 0x00
byte[1] = opcode & 0xFF = 0x82
byte[2] = addr & 0xFF = 0x16
byte[3] = (addr >> 8) & 0xFF = 0xD8
byte[4] = len & 0xFF = 0x04
byte[5] = (len >> 8) & 0xFF = 0x00
byte[6] = (addr >> 16) & 0xFF = 0x00
byte[7] = 0x00

```

Resultado: 00 82 16 D8 04 00 00 00

Paquete 2 — DATOS (wValue = 0x0B00)

Payload de 4 bytes relleno con ceros hasta 8:

```

byte[0] = 0x00
byte[1] = 0x78 (constante del protocolo)
byte[2] = canal (0x10..0x17)
byte[3] = 0xFF encender / 0x00 apagar
byte[4..7] = 0x00

```

Esperar ~1 ms entre paquetes.

Canales

Canal	Luz	Archivo de salida
0x10	Blanca	white.jpg
0x11	Polarizada cruzada (XPL)	negative.jpg
0x12	Polarizada paralela (PPL)	positive.jpg
0x13	Wood	wsg.jpg

0x14	UV	uv.jpg
0x15	Azul	blue.jpg
0x16	Marrón	orange.jpg
0x17	Roja	red.jpg

Apagar todo: canal 0x13 con valor 0x00 .

Secuencia de captura

Orden exacto del original — respétalo, los algoritmos de análisis esperan este conjunto de imágenes:

0x10 → 0x11 → 0x12 → 0x13 → 0x14 → 0x15 → 0x16 → 0x17

Tiempos (no los acortes; son la estabilización de los LEDs):

Momento	ms
Antes del primer disparo	1500
Entre luces	3600
Después de cada foto	2000

Ciclo por luz: encender → esperar → `captureStill()` → esperar → siguiente. Al terminar, **apagar las luces siempre**, incluso si hubo error o el usuario canceló. Envolver la secuencia en `try/finally` .

Todo esto en corrutina fuera del hilo principal: la secuencia completa dura unos 45 segundos y hay `sleep` de por medio.

4. Librerías nativas — CRÍTICO

`UVCCamera.<clinit>` carga, en este orden:

```
System.loadLibrary("jpeg-turbo1500");
System.loadLibrary("usb100");
System.loadLibrary("uvc");
System.loadLibrary("UVCCamera");
```

Si falta cualquiera → `UnsatisfiedLinkError` en el inicializador estático y `FATAL EXCEPTION: CameraThread`. **Un intento anterior falló exactamente por omitir `libjpeg-turbo1500.so`**, que además es dependencia declarada (`DT_NEEDED`) de `libuvc.so`.

Copiar a `app/src/main/jniLibs/<abi>/`, extraídas del APK original:

```
libjpeg-turbo1500.so    ← la que se olvidó
libusb100.so
libuvc.so
libUVCCamera.so
libc++_shared.so
libSkinDetect.so        ← motor de análisis (sección 5)
```

Incluir **ambas ABIs**: `arm64-v8a` y `armeabi-v7a`. El APK original trae las dos.

```
ndk { abiFilters 'arm64-v8a', 'armeabi-v7a' }
```

`libUVCCamera.so` es un fork del proyecto de saki4510t con `xu_write` añadido. Las clases Java `com.serenegiant.*` deben conservar ese nombre de paquete: el JNI resuelve por nombre.

5. Motor de análisis (SDK licenciado)

El análisis lo hace **Gushang SkinDetect**, un SDK de terceros que el propietario ya tiene licenciado. Corre **en el dispositivo**: no hay que subir nada a ningún servidor. Ese es justamente el punto del proyecto.

Integración

- Copiar el paquete `com.gushang.*` completo del APK original.
- **No renombrar el paquete.** El JNI resuelve por símbolo:
`Java_com_gushang_skindetect_JniInterface_skinOilContent`. Si lo renombas, no enlaza.
- Copiar `libSkinDetect.so`.
- Copiar los modelos a `assets/`: `pyramidbox.nb`, `facekeypoints.nb`,
`maskclassifier.nb`.

Activación

```
int result = JNIInterface.register(anyString);    // != 0 → activado
```

La licencia vive en un **archivo del sistema en** `/sdcard/skindetect`, leído por el `.so`. No está atada al nombre del paquete ni a la firma de la app. El argumento de `register()` es irrelevante (el original pasa un string aleatorio).

Llamar a `register()` una vez al arrancar, antes de cualquier análisis. Si devuelve 0, mostrar un mensaje claro de “licencia no encontrada” en vez de continuar y fallar más adelante.

Requiere permiso de lectura sobre almacenamiento externo.

Calibración

El `.so` busca `calib_value_1.txt`, `calib_value_2.txt` y `calib_value_3.txt`, y expone `skinDataCalibration` / `skinDataVerification`. Si esos archivos existen en la tablet, **preservarlos**: afectan la precisión de los valores.

Firmas disponibles

Todas reciben `(String, String, byte[], int, int)`:

<code>skinOilContent</code>	<code>skinWaterContent</code>	<code>skinWhiteness</code>	<code>skinElasticity</code>
<code>skinSensitivity</code>	<code>skinAcne</code>	<code>skinBlackheads</code>	<code>skinScars</code>
<code>skinCuticle</code>	<code>skinSpotColor</code>	<code>skinHair</code>	<code>skinExudates</code>
<code>skinHeavyMetal</code>	<code>skinAcneCuticle</code>	<code>skinAcneInflammation</code>	
			→ float
<code>skinHeatMap</code>	<code>skinThreeDImage</code>		→ int
<code>skinTest(int, String, String)</code>			→ float

Los dos primeros parámetros son rutas de imagen de entrada y salida; el `byte[]` con los dos enteros son los landmarks faciales y sus dimensiones.

El SDK también soporta análisis de cuero cabelludo (endpoints `/ai/v1/scalp/...`), capacidad que la app original no expone. Fuera de alcance por ahora, pero tenerlo presente.

6. Stack y estructura

- Kotlin + Jetpack Compose
- Corrutinas para captura y análisis (nunca en el hilo principal)
- Room para pacientes, sesiones e histórico

- MediaPipe Face Landmarker para los puntos faciales

- minSdk 26, targetSdk 34

Estructura sugerida:

hardware/	UvcSession (única dueña del USB), controlador de luces, secuencia
analysis/	envoltorio del SDK gushang, mapeo de indicadores
data/	Room: Patient, Session, Indicator, Product
ui/	pantallas: pacientes, captura, resultados, comparación, reporte
report/	generación de PDF

7. Funcionalidad

Pacientes: alta con nombre, edad y sexo (la edad real importa: el motor la usa). Historial por paciente y comparación entre sesiones.

Captura: preview en vivo, guía de posicionamiento (línea media de la nariz), indicación de cerrar los ojos, botón único que dispara la secuencia de 8, barra de progreso mostrando qué luz va, y revisión de las 8 fotos antes de analizar con opción de repetir.

Resultados: los indicadores en dos grupos.

- *Superficie:* sebo, poros, manchas, arrugas, acné, puntos negros, ojeras, tono
- *Profundo:* sensibilidad, manchas UV, acné UV, colágeno, pigmentación, porfirinas

Con imagen anotada, valor cuantitativo, nivel de severidad en 5 grados (mínimo → urgente) y comparativa por zonas (izquierda, derecha, zona T).

Reporte: PDF con logo de la clínica, exportable y compartible. Nada se sube a ningún servidor.

Idioma: español, revisado. La app original tiene traducciones automáticas pésimas ("Prueba de la piel indicador", "combinación Valoración") — no las copies.

8. Encuadre médico

Esto es importante y no es opcional.

La máquina hace **análisis cosmético de piel**, no diagnóstico. La app debe declararlo de forma visible en la pantalla de resultados y en el PDF.

- No usar lenguaje diagnóstico ("melanoma", "carcinoma", "lesión sospechosa").
- No sugerir tratamientos médicos ni fármacos.
- Incluir una nota indicando que cualquier lesión sospechosa requiere evaluación dermatológica presencial.

La usuaria es médico y sabrá interpretar, pero el reporte puede terminar en manos del paciente.

9. Errores conocidos a evitar

Aprendidos de un intento previo que falló:

1. **No** abrir una segunda `UsbDeviceConnection` para las luces. → cámara muerta.
 2. **No** olvidar `libjpeg-turbo1500.so` . → `FATAL EXCEPTION: CameraThread` .
 3. **No** empaquetar una sola ABI.
 4. **No** llamar a `applyLightMode` antes de que la cámara esté abierta.
 5. **No** dejar las luces encendidas al salir o al fallar.
 6. **No** renombrar los paquetes `com.gushang.*` ni `com.serenegiant.*` .
 7. **No** bloquear el hilo principal con los `sleep` de la secuencia.
-

10. Verificación por etapas

No avanzar a la siguiente hasta que la anterior pase.

1. **Carga de nativas.** App arranca sin `UnsatisfiedLinkError` .
2. **Cámara.** Preview en vivo estable.
3. **Luces.** Pantalla de prueba con 8 botones: cada uno enciende su luz y el preview sigue vivo. *Si el preview muere al encender una luz, hay dos conexiones USB.*
4. **Licencia.** `register()` devuelve distinto de 0.
5. **Secuencia.** Las 8 fotos se generan con los nombres correctos.
6. **Análisis.** El SDK devuelve valores coherentes sobre esas 8 imágenes.

7. Interfaz y reporte.

Al probar en la tablet, filtrar logcat por el paquete propio, `Mj008` , `UVCCamera` y `AndroidRuntime` .